Building the 9,000-profile pool by combining multiple GSS cross-sections

Table of contents


```
# Required packages
# install.packages(c("tidyverse", "haven", "labelled", "fs", "glue", "here"))
# remotes::install_github("kjhealy/gssr")

library(tidyverse)
library(gssr)       # GSS data (cumulative gss_all + gss_get_yr)
library(haven)      # labelled data handling
library(labelled)   # label utilities
```

```r
library(glue)       # string interpolation
library(here)       # project paths
library(fs)         # file paths

set.seed(2026)
```

1 Overview

This script builds a 9,000-profile demographic pool for LLM simulation by pooling multiple recent GSS cross-sectional waves rather than duplicating respondents by weight from a single year.

Key design decisions:

- **Years used**: The 4 most recent distinct cross-sectional GSS waves (2016, 2018, 2022, 2024). GSS was not fielded in 2020 (COVID); 2021 was a panel recontact of prior respondents, not a fresh cross-section.
- **Weight reconciliation**: Each year's `wtssps` is normalized within year (divides by that year's weight sum), so each wave contributes equal total weight to the sampling lottery while preserving within-year relative weights.
- **Sampling**: Weighted without replacement (N = 9,000). If the 4-year pool falls short of 9,000 complete cases, the script extends to 2014 automatically and warns.
- **Output**: Demographic dataset only. Prompt strings are not generated here — that step is handled by a collaborator in a separate project.

This approach avoids the artificial duplication inherent in weight-cloning from a single year (where high-weight respondents would appear 3–4×), replacing it with genuinely different individuals surveyed in different years.

2 Variable selection

Same 19 GSS variables as `build_realistic_clone_profiles.qmd`, minus prompt output.

```r
selected_vars <- c(
  # weighting and identifiers
  "year", "id", "wtssps",
  # core demographics
  "age", "sex", "racecen1", "hispanic",
  "degree", "income16", "hompop", "class",
  "region", "srcbelt",
```

```
  # politics
  "partyid", "polviews",
  # religion
  "relig", "reborn", "reliten",
  # opinion
  "consci"
)

# Variables where NA means "drop respondent"
required_vars <- c(
  "wtssps",
  "age", "sex", "racecen1", "hispanic",
  "degree", "income16", "class",
  "region",
  "partyid", "polviews",
  "relig"
)

# Variables where NA is acceptable (handled downstream)
optional_vars <- c("hompop", "reborn", "reliten", "consci", "srcbelt")

numeric_vars <- c("age", "hompop", "wtssps")
factor_vars  <- setdiff(selected_vars, c("year", "id", numeric_vars))
```

3 Step 1. Load the cumulative GSS and select target years

We use `gss_all` (the full cumulative GSS file shipped with the `gssr` package) and filter to our target years. This is more efficient than four separate `gss_get_yr()` calls.

```
# The initial target: 4 most recent true cross-sectional waves
target_years <- c(2018, 2021, 2022, 2024)

data(gss_all)

gss_multi <- gss_all |>
  filter(year %in% target_years) |>
  select(any_of(selected_vars))

# Year audit - verify all target years loaded and check raw N per year
cat("Raw respondent counts by year:\n")
```

Raw respondent counts by year:

```
print(table(gss_multi$year))
```

```
2018 2021 2022 2024
2348 4032 3544 3309
```

```
cat(glue("\nTotal raw respondents across {length(target_years)} years: {nrow(gss_multi)}\n"))
```

```
Total raw respondents across 4 years: 13233
```

4 Step 2. Select, type-convert, and recode GSS missing labels

```
# Convert weight and numeric vars; attach factor labels to all others
gss_sel <- gss_multi |>
  mutate(across(all_of(numeric_vars), ~ as.numeric(haven::zap_labels(.x)))) |>
  mutate(across(all_of(factor_vars),  ~ haven::as_factor(.x)))


gss_missing_labels <- c(
  "don't know", "no answer", "skipped on web", "iap",
  "refused", "not imputable", "dk, na, iap",
  "uncodeable",
  "not available in this release", "not available in this year",
  "see codebook"
)

gss_sel <- gss_sel |>
  mutate(across(where(is.factor), ~ {
    chr <- as.character(.x)
    chr[tolower(chr) %in% gss_missing_labels] <- NA
    factor(chr, levels = setdiff(levels(.x), gss_missing_labels))
  }))

# Roll up Asian sub-categories
asian_categories <- c(
  "asian indian", "chinese", "filipino", "japanese",
```

```
  "korean", "vietnamese", "other asian"
)

gss_sel <- gss_sel |>
  mutate(
    racecen1 = as.character(racecen1),
    racecen1 = case_when(
      racecen1 %in% asian_categories                                                    ~ "asian"
      is.na(racecen1) & !is.na(hispanic) & as.character(hispanic) != "not hispanic" ~ "hispa
      TRUE                                                                              ~ racec
    ),
    racecen1 = factor(racecen1)
  )

# check
gss_sel |>
  summarise(across(everything(),
                   ~ mean(is.na(.x), na.rm = FALSE),
                   .names = "{.col}_na_share")) |>
  tidyr::pivot_longer(everything(),
                      names_to = "variable",
                      values_to = "na_share") |>
  mutate(variable = stringr::str_remove(variable, "_na_share"),
         na_share = scales::percent(na_share, accuracy = 0.1))
```

```
# A tibble: 19 x 2
   variable na_share
   <chr>    <chr>
 1 year     0.0%
 2 id       0.0%
 3 wtssps   0.0%
 4 age      4.9%
 5 sex      1.0%
 6 racecen1 0.9%
 7 hispanic 0.9%
 8 degree   0.2%
 9 income16 11.6%
10 hompop   27.2%
11 class    0.8%
12 region   0.0%
13 srcbelt  25.4%
14 partyid  1.1%
```

```
15 polviews 3.3%
16 relig    1.5%
17 reborn   12.8%
18 reliten  58.5%
19 consci   35.0%
```

5 Step 3. Remove missing data for required variables

```
gss_complete <- gss_sel |>
  drop_na(all_of(required_vars))

# Per-year complete-case audit
cat("Complete cases by year (after required-var filter):\n")
```

```
Complete cases by year (after required-var filter):
```

```
gss_complete |>
  count(year) |>
  mutate(pct_of_raw = round(100 * n / table(gss_multi$year)[as.character(year)], 1)) |>
  print()
```

```
# A tibble: 4 x 3
  year          n pct_of_raw
  <dbl+lbl> <int> <table[1d]>
1 2018       2039 86.8
2 2021       3237 80.3
3 2022       2832 79.9
4 2024       2717 82.1
```

```
cat(glue(
  "\nTotal eligible pool: {nrow(gss_complete)} respondents ",
  "({round(100 * nrow(gss_complete) / nrow(gss_sel), 1)}% of {nrow(gss_sel)} multi-year rows)
))
```

```
Total eligible pool: 10825 respondents (81.8% of 13233 multi-year rows)
```

```r
# Safety check: if pool is still under 9000, extend to 2014
if (nrow(gss_complete) < 9000) {
  warning(
    "Pool of ", nrow(gss_complete), " complete cases is below the target N = 9000. ",
    "Extending to 2014. Re-run from Step 1 with target_years <- c(2014, 2016, 2018, 2022, 202
  )
}

# Per-variable NA rates among kept respondents (optional variables only)
cat("\nNA rates on optional variables (kept respondents):\n")
```

```
NA rates on optional variables (kept respondents):
```

```r
gss_complete |>
  summarise(across(all_of(optional_vars), ~ round(100 * mean(is.na(.x)), 1))) |>
  pivot_longer(everything(), names_to = "variable", values_to = "pct_na") |>
  print()
```

```
# A tibble: 5 x 2
  variable pct_na
  <chr>     <dbl>
1 hompop     26
2 reborn     11.3
3 reliten    57.6
4 consci     34.6
5 srcbelt    25.6
```

6 Step 4. Within-year weight normalization

Each year's **wtssps** values sum to that year's effective sample size, which differs across waves. Normalizing within year ensures each wave contributes equal total weight to the sampling lottery, while preserving the relative ordering of respondents within each year.

```r
gss_complete <- gss_complete |>
  group_by(year) |>
  mutate(w_pooled = wtssps / sum(wtssps, na.rm = TRUE)) |>
  ungroup()
```

```r
# Sanity check: each year's normalized weights should sum to 1.0
cat("Sum of normalized weights by year (should be 1.0 each):\n")
```

```
Sum of normalized weights by year (should be 1.0 each):
```

```r
gss_complete |>
  group_by(year) |>
  summarise(w_sum = round(sum(w_pooled, na.rm = TRUE), 6)) |>
  print()
```

```
# A tibble: 4 x 2
  year      w_sum
  <dbl+lbl> <dbl>
1 2018          1
2 2021          1
3 2022          1
4 2024          1
```

7 Step 5. Weighted sampling without replacement (N = 9,000)

```r
target_n <- 9000

# Guard: pool must be large enough for without-replacement sampling
stopifnot(
  "Pool too small for without-replacement sampling - extend target_years to include 2014" =
    nrow(gss_complete) >= target_n
)

profiles <- gss_complete |>
  slice_sample(n = target_n, weight_by = w_pooled, replace = FALSE) |>
  select(-w_pooled) |>
  mutate(profile_id = sprintf("p%05d", row_number())) |>
  select(profile_id, everything())

glue("Profile pool built: N = {nrow(profiles)} unique individuals ",
     "drawn from {nrow(gss_complete)} eligible respondents across ",
     "{n_distinct(profiles$year)} GSS waves.")
```

```
Profile pool built: N = 9000 unique individuals drawn from 10825 eligible respondents across
```

8 Step 6. Validate the profile pool

8.1 Year balance

Normalized within-year weights means each year contributes equal total sampling probability. In expectation, the sample should be split roughly equally across years (~2,250 per year for a 4-year pool).

```
cat("Sample composition by GSS year:\n")
```

```
Sample composition by GSS year:
```

```
profiles |>
  count(year) |>
  mutate(pct = round(100 * n / sum(n), 1)) |>
  print()
```

```
# A tibble: 4 x 3
  year          n   pct
  <dbl+lbl> <int> <dbl>
1 2018       1867  20.7
2 2021       2687  29.9
3 2022       2214  24.6
4 2024       2232  24.8
```

8.2 Uniqueness check

```
n_unique_pairs <- nrow(distinct(profiles, year, id))
cat(glue(
  "Unique (year, id) pairs: {n_unique_pairs} of {nrow(profiles)} ",
  "({if (n_unique_pairs == nrow(profiles)) 'PASS - no duplicates' else 'FAIL - duplicates pr
))
```

```
Unique (year, id) pairs: 9000 of 9000 (PASS - no duplicates)
```

8.3 Marginal distributions vs. weighted 2024 GSS

We use the 2024 wave (most recent) as the reference population. The pooled sample should approximate 2024-weighted GSS marginals on key variables; deviations are expected because older waves represent slightly different demographic compositions.

```
gss_2024 <- gss_complete |> filter(year == 2024)

validate_marginal <- function(var) {
  src <- gss_2024 |>
    count(.data[[var]], wt = wtssps) |>
    mutate(prop_gss2024 = n / sum(n)) |>
    select(-n)
  prof <- profiles |>
    count(.data[[var]]) |>
    mutate(prop_pool = n / sum(n)) |>
    select(-n)
  full_join(src, prof, by = var) |>
    mutate(diff = round(prop_pool - prop_gss2024, 4),
           prop_gss2024 = round(prop_gss2024, 4),
           prop_pool    = round(prop_pool, 4))
}

vars_to_check <- c("sex", "racecen1", "degree", "partyid", "polviews", "region")

walk(vars_to_check, \(v) {
  cat("\n----", v, "----\n")
  print(validate_marginal(v))
})
```

```
---- sex ----
# A tibble: 2 x 4
  sex    prop_gss2024 prop_pool    diff
  <fct>         <dbl>     <dbl>   <dbl>
1 male          0.503     0.473 -0.0301
2 female        0.497     0.527  0.0301

---- racecen1 ----
# A tibble: 7 x 4
  racecen1                          prop_gss2024 prop_pool    diff
  <fct>                                    <dbl>     <dbl>   <dbl>
```

```
1 american indian or alaska native       0.0148    0.0132 -0.0016
2 asian                                  0.0629    0.0398 -0.0231
3 black or african american              0.128     0.137   0.0091
4 hispanic                               0.0573    0.0454 -0.0119
5 other pacific islander                 0.0047    0.0023 -0.0024
6 some other race                        0.0094    0.0089 -0.0005
7 white                                  0.723     0.754   0.0304

---- degree ----
# A tibble: 5 x 4
  degree                   prop_gss2024 prop_pool    diff
  <fct>                           <dbl>     <dbl>   <dbl>
1 less than high school          0.0887    0.0772 -0.0115
2 high school                    0.469     0.451  -0.0173
3 associate/junior college       0.0923    0.0951  0.0028
4 bachelor's                     0.212     0.227   0.0152
5 graduate                       0.138     0.149   0.0108

---- partyid ----
# A tibble: 8 x 4
  partyid                            prop_gss2024 prop_pool    diff
  <fct>                                     <dbl>     <dbl>   <dbl>
1 strong democrat                          0.155     0.180   0.0243
2 not very strong democrat                 0.110     0.138   0.0282
3 independent, close to democrat           0.112     0.122   0.0101
4 independent (neither, no response)       0.232     0.192  -0.0397
5 independent, close to republican         0.0895    0.0964  0.0069
6 not very strong republican               0.119     0.112  -0.0067
7 strong republican                        0.156     0.131  -0.0241
8 other party                              0.027     0.0281  0.0011

---- polviews ----
# A tibble: 7 x 4
  polviews                     prop_gss2024 prop_pool    diff
  <fct>                               <dbl>     <dbl>   <dbl>
1 extremely liberal                  0.0396    0.0511  0.0115
2 liberal                            0.136     0.144   0.0083
3 slightly liberal                   0.117     0.122   0.0055
4 moderate, middle of the road       0.360     0.354  -0.0064
5 slightly conservative              0.116     0.125   0.0091
6 conservative                       0.167     0.157  -0.0104
7 extremely conservative             0.0643    0.0468 -0.0175
```

```
---- region ----
# A tibble: 4 x 4
  region    prop_gss2024 prop_pool    diff
  <fct>            <dbl>     <dbl>   <dbl>
1 northeast        0.178     0.159 -0.0191
2 midwest          0.213     0.232  0.0187
3 south            0.380     0.384  0.0045
4 west             0.229     0.225 -0.0041
```

9 Step 7. Save output

```
out_path <- here("clone_profiles", "profiles.csv")
write_csv(profiles, out_path)

cat(glue(
  "Saved: {out_path}\n",
  "  Rows    : {nrow(profiles)}\n",
  "  Columns : {ncol(profiles)}\n",
  "  Years   : {paste(sort(unique(profiles$year)), collapse = ', ')}\n"
))
```

```
Saved: /Users/jan/Documents/git/llm_predictions_megastudy/clone_profiles/profiles.csv
Rows    : 9000
Columns : 20
Years   : 2018, 2021, 2022, 2024
```